

QuantLib Derivatives Pricing

Supplementary Technical Slides

Quantitative Analytics Team

August 2026

Appendix A

Mathematical Foundations

Stochastic Calculus Essentials

Brownian Motion: Definition

Standard Brownian Motion W_t

A stochastic process with:

- ① $W_0 = 0$
- ② Independent increments: $W_t - W_s \perp W_s - W_u$ for $t > s > u$
- ③ Normally distributed: $W_t - W_s \sim N(0, t - s)$
- ④ Continuous paths

Key Properties:

- $\mathbb{E}[W_t] = 0$
- $\text{Var}[W_t] = t$
- $\mathbb{E}[W_t W_s] = \min(t, s)$
- Paths are nowhere differentiable

Itô's Lemma

Itô's Formula

For $f(t, X_t)$ where $dX_t = \mu dt + \sigma dW_t$:

$$df = \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial x} dX_t + \frac{1}{2} \frac{\partial^2 f}{\partial x^2} \sigma^2 dt$$

Example: Geometric Brownian Motion

Let S_t follow $dS_t = \mu S_t dt + \sigma S_t dW_t$. Apply Itô to $f(S) = \ln S$:

$$d \ln S = \frac{1}{S} dS - \frac{1}{2} \frac{1}{S^2} \sigma^2 S^2 dt = \left(\mu - \frac{\sigma^2}{2} \right) dt + \sigma dW_t$$

Therefore: $S_T = S_0 \exp \left[\left(\mu - \frac{\sigma^2}{2} \right) T + \sigma W_T \right]$

Risk-Neutral Pricing

Fundamental Theorem of Asset Pricing

No-arbitrage $\Leftrightarrow$ Exists a probability measure $\mathbb{Q}$ such that discounted asset prices are martingales.

Pricing Formula:

$$V_0 = \mathbb{E}^{\mathbb{Q}} \left[e^{-\int_0^T r_s ds} \cdot \text{Payoff}_T \right]$$

Change of Measure (Girsanov):

- Under $\mathbb{P}$: $dS = \mu S dt + \sigma S dW^{\mathbb{P}}$
- Under $\mathbb{Q}$: $dS = rS dt + \sigma S dW^{\mathbb{Q}}$
- $dW^{\mathbb{Q}} = dW^{\mathbb{P}} + \frac{\mu - r}{\sigma} dt$

Ornstein-Uhlenbeck Process

OU Process

$$dx_t = -ax_t dt + \sigma dW_t$$

Solution:

$$x_t = x_0 e^{-at} + \sigma \int_0^t e^{-a(t-s)} dW_s$$

Properties:

- Mean: $\mathbb{E}[x_t] = x_0 e^{-at}$
- Variance: $\text{Var}[x_t] = \frac{\sigma^2}{2a} (1 - e^{-2at})$
- Stationary distribution: $N(0, \frac{\sigma^2}{2a})$
- Auto-correlation: $\text{Corr}(x_t, x_s) = e^{-a|t-s|}$

Hull-White: Add time-dependent mean $\theta(t)$

Affine Term Structure Models

Affine Property

Discount bond prices have the form:

$$P(t, T) = e^{A(t, T) - B(t, T)r_t}$$

For Hull-White:

$$B(t, T) = \frac{1 - e^{-a(T-t)}}{a}$$

$$A(t, T) = \int_t^T \theta(s)B(s, T) ds - \frac{\sigma^2}{2} \int_t^T B(s, T)^2 ds$$

Implications:

- Analytical bond and swaption prices
- Efficient calibration
- Foundation for tree methods

Multi-Factor Models: Correlated Brownians

Correlated Brownian Motions:

Given correlation matrix $\mathbf{R}$, generate correlated increments:

$$\begin{pmatrix} dW_1 \\ dW_2 \\ dW_3 \end{pmatrix} = \mathbf{L} \cdot \begin{pmatrix} dZ_1 \\ dZ_2 \\ dZ_3 \end{pmatrix}$$

Where $\mathbf{L}$ is the Cholesky decomposition: $\mathbf{R} = \mathbf{L}\mathbf{L}^T$

Example: $\rho_{12} = 0.25$, $\rho_{13} = -0.15$, $\rho_{23} = -0.30$

$$\mathbf{L} = \begin{pmatrix} 1 & 0 & 0 \\ 0.25 & 0.968 & 0 \\ -0.15 & -0.271 & 0.951 \end{pmatrix}$$

Variance Calculation for Joint Processes

For state $\mathbf{x}_t$:

$$\text{Cov}[\mathbf{x}_{t+h}, \mathbf{x}_{t+h}] = \int_0^h \mathbf{K}(s) \cdot \mathbf{R} \cdot \mathbf{K}(s)^T ds$$

Where $\mathbf{K}(s)$ is the kernel matrix mapping Brownians to state.

Numerical Integration:

$$\int_0^h f(s) ds \approx \frac{h}{2} \sum_{i=1}^n w_i f\left(\frac{h}{2}(x_i + 1)\right)$$

Using Gauss-Legendre nodes x_i and weights w_i .

Our Implementation: 16-point Gauss-Legendre quadrature

Change of Numéraire

Numéraire Change

When changing from numéraire N_1 to N_2 :

$$\left. \frac{dQ^{N_2}}{dQ^{N_1}} \right|_{\mathcal{F}_T} = \frac{N_2(T)/N_2(0)}{N_1(T)/N_1(0)}$$

FX Pricing Application:

Under USD money-market measure:

- USD bank account is numéraire
- EUR rates get quanto adjustment
- FX drift includes rate differential

$$d \ln X = (r^{USD} - r^{EUR} - \frac{1}{2}\sigma_X^2) dt + \sigma_X dW_X$$

Quanto Adjustment Derivation

Setting: Price a EUR asset S^{EUR} from USD perspective

Under EUR measure:

$$dS^{EUR} = r^{EUR} S^{EUR} dt + \sigma_S S^{EUR} dW_S^{EUR}$$

Converting to USD measure:

$$dS^{EUR} = (r^{EUR} - \rho_{S,X} \sigma_S \sigma_X) S^{EUR} dt + \sigma_S S^{EUR} dW_S^{USD}$$

Intuition:

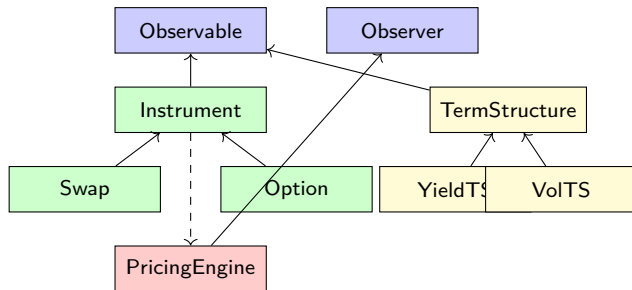
- If $\rho_{S,X} > 0$: Asset rises when EUR appreciates
- Extra expected USD return from FX
- Must subtract to stay risk-neutral

Appendix B

QuantLib Deep Dive

Internal Architecture and APIs

QuantLib Class Hierarchy



Key Patterns:

- Observer/Observable for lazy evaluation
- Instruments are priced by Engines
- Term structures hold market data

Date and Calendar Handling

```
import QuantLib as ql

# Date construction
d1 = ql.Date(15, 8, 2026) # Day, Month, Year
d2 = ql.Date(15, ql.August, 2026) # Using enum
d3 = ql.DateParser.parseISO("2026-08-15")

# Date arithmetic
d4 = d1 + ql.Period(3, ql.Months)
d5 = d1 + 30 # Add 30 calendar days

# Calendar operations
cal = ql.UnitedStates(ql.UnitedStates.Settlement)
if cal.isBusinessDay(d1):
    print("Business day")

# Advance by business days
spot = cal.advance(d1, 2, ql.Days)

# Joint calendar (requires both to be open)
joint = ql.JointCalendar(ql.TARGET(),
    ql.UnitedStates(ql.UnitedStates.Settlement),
    ql.JoinHolidays)
```

Index Classes

```
# Overnight indices
sofr = ql.Sofr()
estr = ql.Estr()
sonia = ql.Sonia()

# Link to curve
curve_handle = ql.YieldTermStructureHandle(curve)
sofr_with_curve = ql.Sofr(curve_handle)

# Get fixings
fixing_date = ql.Date(10, 8, 2026)
rate = sofr.fixing(fixing_date)

# Add historical fixings (for backward-looking)
ql.IndexManager.instance().clearHistories()
sofr.addFixing(ql.Date(1, 8, 2026), 0.04)
sofr.addFixing(ql.Date(2, 8, 2026), 0.0402)
```

Yield Curve Construction Details

```
# Rate helpers
helpers = []

# Deposit helper (for short end)
helpers.append(ql.DepositRateHelper(
    ql.QuoteHandle(ql.SimpleQuote(0.041)), # Rate
    ql.Period(1, ql.Days), # Tenor
    0, # Settlement days
    ql.UnitedStates(ql.UnitedStates.Settlement),
    ql.ModifiedFollowing, # Convention
    False, # End of month
    ql.Actual360() # Day count
))
```

```
# OIS helpers
for tenor, rate in [("1Y", 0.039), ("5Y", 0.037)]:
    helpers.append(ql.OISRateHelper(
        2, ql.Period(tenor),
        ql.QuoteHandle(ql.SimpleQuote(rate)),
        ql.Sofr()
    ))
```

```
# Build curve
```

```
curve = ql.PiecewiseLogLinearDiscount(Today, helpers, ql.Actual365Fixed(0))
```


OvernightIndexedSwap Instrument

```
# Create OIS swap
schedule = ql.Schedule(
    start_date, end_date,
    ql.Period(ql.Semiannual), # Payment frequency
    calendar,
    ql.ModifiedFollowing, # Business day convention
    ql.ModifiedFollowing,
    ql.DateGeneration.Backward,
    False # End of month
)

swap = ql.OvernightIndexedSwap(
    ql.Swap.Payer, # Type
    100_000_000.0, # Notional
    schedule,
    0.04, # Fixed rate
    ql.Actual360(), # Fixed day count
    ql.Sofr(curve_handle), # Index
    0.0, # Spread on floating
    0, # Payment lag
    ql.ModifiedFollowing,
    calendar
)
```

Swaption Construction

```
# Create underlying swap
underlying = ql.VanillaSwap(
    ql.Swap.Payer, notional,
    fixed_schedule, strike, day_count,
    float_schedule, sofr_index, 0.0, day_count
)

# European exercise
expiry = ql.Date(15, 8, 2028)
exercise = ql.EuropeanExercise(expiry)

# Bermudan exercise
call_dates = [ql.Date(15, 8, 2028 + i) for i in range(5)]
exercise = ql.BermudanExercise(call_dates)

# Create swaption
swaption = ql.Swaption(underlying, exercise)

# Set pricing engine
engine = ql.TreeSwaptionEngine(hw_model, 50)
swaption.setPricingEngine(engine)

print(f"NPV: {swaption.NPV():.2f}")
```

Hull-White Model Setup

```
# Create model with fixed parameters
mean_reversion = 0.03
volatility = 0.01
model = ql.HullWhite(curve_handle, mean_reversion, volatility)

# Get model parameters
params = model.params()
print(f"a = {params[0]}, sigma = {params[1]}")

# Create calibration helpers
helpers = []
for expiry, tenor, vol_bp in [("2Y", "8Y", 78), ("5Y", "5Y", 80)]:
    helper = ql.SwaptionHelper(
        ql.Period(expiry), ql.Period(tenor),
        ql.QuoteHandle(ql.SimpleQuote(vol_bp * 1e-4)),
        sofr_index, ql.Period("1Y"), ql.Actual360(),
        ql.Actual360(), curve_handle,
        ql.BlackCalibrationHelper.RelativePriceError,
        ql.nullDouble(), 1.0, ql.Normal
    )
    helper.setPricingEngine(ql.JamshidianSwaptionEngine(model))
    helpers.append(helper)
```

Model Calibration

```
# Calibration settings
opt_method = ql.LevenbergMarquardt()
end_criteria = ql.EndCriteria(
    500, # Max iterations
    100, # Max stationary
    1e-10, # Root epsilon
    1e-10, # Function epsilon
    1e-10 # Gradient epsilon
)

# Which parameters to calibrate
# [True, False] means calibrate a, keep sigma fixed
fixed_params = ql.BoolVector([True, False])

# Run calibration
model.calibrate(
    helpers, # Calibration instruments
    opt_method,
    end_criteria,
    ql.PositiveConstraint(), # Parameter constraint
    ql.DoubleVector(), # Weights
    fixed_params
)
```

Calibration Diagnostics

```
# After calibration, check fit quality
for i, helper in enumerate(helpers):
    market_val = helper.marketValue()
    model_val = helper.modelValue()
    error = (model_val - market_val) / market_val

    # Get implied vol
    impl_vol = helper.impliedVolatility(model_val, 1e-6, 100,
                                         1e-6, 2.0)

    print(f"Helper {i}:")
    print(f" Market: {market_val:.6f}")
    print(f" Model: {model_val:.6f}")
    print(f" Error: {error*100:.3f}%")
    print(f" Impl Vol: {impl_vol*10000:.1f}bp")

# Calculate weighted RMS
errors = [(h.modelValue()/h.marketValue() - 1)**2 for h in helpers]
rms = math.sqrt(sum(errors) / len(errors))
print(f"RMS Error: {rms*100:.3f}%")
```

QuantLib Limitations for XCCY

Native XCCY Support:

- `CrossCurrencyBasisSwap` — exists for vanilla XCCY
- `CrossCurrencyBasisSwapRateHelper` — for curve building
- No native **callable** XCCY instrument

Multi-Factor Limitations:

- Tree engines work for 1-2 factors
- No native 3-factor joint tree
- Would need FD solver on 3D grid (slow)
- Monte Carlo is natural choice

Our Solution:

- Use QuantLib for curves, dates, indices
- Custom 3-factor Monte Carlo
- LSM for exercise optimization

Appendix C

Monte Carlo Methods

Simulation Techniques and Variance Reduction

Monte Carlo Basics

Law of Large Numbers

$$\frac{1}{N} \sum_{i=1}^N f(X_i) \xrightarrow{N \rightarrow \infty} \mathbb{E}[f(X)]$$

Standard Error:

$$SE = \frac{\sigma}{\sqrt{N}}$$

95% Confidence Interval:

$$\bar{X} \pm 1.96 \times SE$$

Key Properties:

- Error decreases as $O(N^{-1/2})$
- Independent of dimension (curse of dimensionality avoided)
- Flexible for path-dependent payoffs

Random Number Generation

NumPy PCG64 Generator:

- Permuted Congruential Generator
- Period: 2^{128}
- Excellent statistical properties
- Fast and reproducible

Usage:

```
import numpy as np

# Create generator with seed
rng = np.random.Generator(np.random.PCG64(1729))

# Generate standard normals
Z = rng.standard_normal((paths, dimensions))
```

Reproducibility:

- Same seed $\Rightarrow$ same sequence
- Critical for debugging and validation

Variance Reduction: Antithetic Variates

Antithetic Sampling

If $\hat{\theta}_1 = f(Z)$ and $\hat{\theta}_2 = f(-Z)$ are negatively correlated, then:

$$\text{Var} \left[\frac{\hat{\theta}_1 + \hat{\theta}_2}{2} \right] < \text{Var}[\hat{\theta}_1]$$

Implementation:

```
def antithetic_normals(rng, paths, dims):  
    half = (paths + 1) // 2  
    draws = rng.standard_normal((half, dims))  
    return np.concatenate([draws, -draws])[:paths]
```

Benefits:

- Forces mean of normals to be exactly 0
- Reduces variance for most payoffs
- No additional cost per sample

Variance Reduction: Control Variates

Control Variate Method

Use a correlated variable Y with known mean μ_Y :

$$\hat{\theta}_{CV} = \hat{\theta} - \beta(\bar{Y} - \mu_Y)$$

Optimal β :

$$\beta^* = \frac{\text{Cov}[\hat{\theta}, Y]}{\text{Var}[Y]}$$

Applications in Derivatives:

- Use underlying asset as control
- Use European price for Bermudan
- Use delta-hedged portfolio

Quasi-Monte Carlo (QMC)

Low-Discrepancy Sequences:

- Sobol, Halton, Niederreiter
- Fill space more uniformly than random
- Error $O(N^{-1})$ vs $O(N^{-1/2})$

Trade-offs:

- Better for smooth integrands
- Dimension-dependent effectiveness
- Standard error not directly computable

Not used in our implementation:

- Standard MC simpler to implement
- Antithetic sufficient for our accuracy needs
- QMC would be future enhancement

Path Generation: Euler vs Exact

Euler Discretization:

$$x_{t+\Delta t} = x_t + \mu(x_t)\Delta t + \sigma(x_t)\sqrt{\Delta t} Z$$

Exact Simulation (OU process):

$$x_{t+h} = x_t e^{-ah} + \sigma \int_0^h e^{-a(h-s)} dW_s$$

Our Approach: Exact OU Kernels

- Hull-White factors use exact solution
- No discretization bias
- Larger time steps possible
- Covariance computed exactly

State Evolution Implementation

```
@dataclass
class StepTransition:
    matrix: np.ndarray # Transition matrix A
    offset: np.ndarray # Deterministic drift b
    covariance_root: np.ndarray # sqrt(Sigma)

def evolve_state(state, transition, normals):
    """
     $x_{t+h} = A * x_t + b + L * Z$ 

    state: (paths, 5) current state
    transition: StepTransition object
    normals: (paths, 5) standard normals

    Returns: (paths, 5) new state
    """
    return (state @ transition.matrix.T
            + transition.offset
            + normals @ transition.covariance_root.T)
```

Cash Flow Valuation

For each simulated path:

Fixed Cash Flow:

$$PV = \text{Amount} \times e^{-I_{USD}(t_{pay})}$$

Floating Cash Flow (compounded overnight):

$$\text{Amount} = N \times \left(e^{I_{EUR}(t_{end}) - I_{EUR}(t_{start}) + \text{basis}} - 1 \right)$$

$$PV = \text{Amount} \times X_{t_{pay}} \times e^{-I_{USD}(t_{pay})}$$

FX Conversion: EUR amounts converted at simulated FX rate

Notional Exchanges:

- Initial: Net of notionals at spot
- Final: Net of notionals at terminal FX

LSM: Basis Function Selection

State Variables: $(x_{USD}, x_{EUR}, \ln X - \ln X_0)$

Polynomial Basis (10 functions):

$$\phi = (1, x, y, z, x^2, y^2, z^2, xy, xz, yz)$$

Why This Choice?

- Captures main effects (linear terms)
- Captures curvature (squared terms)
- Captures interactions (cross terms)
- Low-dimensional (avoids overfitting)

Alternatives:

- Laguerre polynomials
- Chebyshev polynomials
- Neural network approximation

LSM: Numerical Stability

Problem: Ill-Conditioned Design Matrix

When state variables have different scales or are correlated:

$$\text{cond}(\mathbf{X}^T \mathbf{X}) \gg 1$$

Solutions:

① Standardization:

$$\tilde{x} = \frac{x - \bar{x}}{\sigma_x}$$

② Ridge Regression:

$$\beta = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$$

Our Implementation:

- Standardize state variables
- Use `np.linalg.lstsq` with `rcond`
- Fall back to ridge if `cond > 1e10`

LSM: Two-Pass Why It Matters

Single-Pass Bias:

- Training on same paths as pricing
- Exercise decision uses future information
- Results in upward bias (too high)

Two-Pass Solution:

- ① **Training Pass:** Learn policy on paths $1 \dots N$
- ② **Pricing Pass:** Apply policy on paths $N+1 \dots 2N$

Properties:

- Pricing paths are independent of training
- No look-ahead bias
- Result is a *lower bound* on true value

Why Lower Bound?

- Suboptimal exercise policy
- True optimal policy would give higher value
- Upper bound requires dual methods

Convergence Diagnostics

Standard Error Analysis:

$$SE = \frac{\sigma}{\sqrt{N}}$$

Expected Behavior:

- Double paths $\Rightarrow$ SE reduces by $\sqrt{2}$
- 10x paths $\Rightarrow$ SE reduces by $\sqrt{10} \approx 3.16$

Test Protocol:

- ① Run with 10K paths, record SE
- ② Run with 40K paths, record SE
- ③ Verify ratio ≈ 2.0

If Ratio is Wrong:

- Implementation bug
- Correlation between paths
- Numerical instability

Appendix D

Testing and Validation

Ensuring Model Correctness

Validation Philosophy

Trust, But Verify

Every model component needs independent validation.

Levels of Testing:

- ① **Unit Tests:** Individual functions
- ② **Integration Tests:** Component interactions
- ③ **Calibration Tests:** Market data fit
- ④ **Model Tests:** Arbitrage conditions
- ⑤ **Convergence Tests:** Numerical accuracy
- ⑥ **Boundary Tests:** Edge cases

Martingale Testing: Theory

Under Risk-Neutral Measure:

Discounted tradeable assets must be martingales:

$$\mathbb{E}^{\mathbb{Q}}[e^{-\int_0^T r_s ds} \cdot A_T] = A_0$$

Test Assets:

- ① **USD Zero Coupon:** $\mathbb{E}[e^{-\int r^{USD}}] = P^{USD}(0, T)$
- ② **EUR Zero in USD:** $\mathbb{E}[e^{-\int r^{USD}} X_T] = X_0 P_{eff}^{EUR}(0, T)$
- ③ **EUR Bank Account:** $\mathbb{E}[e^{-\int r^{USD}} X_T e^{\int r^{EUR}}] = X_0$

Statistical Test:

$$z = \frac{\bar{X}_{MC} - \text{Target}}{SE_{MC}}$$

Pass if $|z| < 3.5$ (approximate 99.95% confidence)

Monotonicity Testing

Economic Relationships That Must Hold:

- $\uparrow$ Volatility $\Rightarrow \uparrow$ Option Value
- $\uparrow$ Non-call Period $\Rightarrow \downarrow$ Option Value
- $\uparrow$ FX Spot (EUR/USD) $\Rightarrow \uparrow$ EUR Leg Value
- $\uparrow$ Interest Rates $\Rightarrow \downarrow$ Long Duration Value

Test Protocol:

- 1 Price with base parameters
- 2 Bump parameter in expected direction
- 3 Verify price moves correctly
- 4 Allow for MC noise (use confidence bounds)

Structural Bounds

Test 1: Option Value Non-Negative

$$V_{callable} \geq V_{non-callable}$$

Test 2: Callable Decomposition

$$V_{callable} = V_{non-callable} + V_{option}$$

Test 3: Option Less Than Notional

$$V_{option} < \text{Notional}$$

Test 4: Leg Consistency

$$V_{USD} + V_{EUR} = V_{non-callable}$$

(Within Monte Carlo error)

Test Implementation Example

```
class TestStructuralBounds:
    """Test fundamental arbitrage bounds."""

    def test_callable_ge_non_callable(self, base_result):
        """Callable >= Non-callable."""
        callable_npv = base_result["valuation"]["callable_npv"]["mean"]
        non_callable = base_result["valuation"]["non_callable_npv"]["mean"]
        option = base_result["valuation"]["embedded_option_value"]["mean"]

        assert option >= 0, f"Option must be >= 0, got {option:,.2f}"

        # Check decomposition
        recomputed = non_callable + option
        assert abs(callable_npv - recomputed) < 1.0, (
            f"Callable ({callable_npv:,.2f}) != "
            f"Non-callable ({non_callable:,.2f}) + Option ({option:,.2f})"
        )
```

Calibration Quality Checks

OIS Curve Bootstrap:

- Each input OIS rate should reprice to $<0.01\text{bp}$
- Interpolation should be smooth
- No negative forward rates

Swaption Calibration:

- Individual error $<5\%$ relative
- RMS error $<5\%$
- Model parameters in reasonable range

FX Vol Calibration:

- Each ATM vol within 25bp
- Term structure monotonic
- Implied sigmas positive

Edge Case Testing

Zero Correlation:

- Set all correlations to 0
- Model should still work
- Results should be sensible

High Correlation:

- Set correlations to extreme (but valid) values
- Check positive semi-definiteness
- Verify no numerical issues

ATM Swap:

- Price with swap at par
- Non-callable value near zero
- Option value dominates

Regression Testing

Golden Reference:

- Store results from validated run
- Compare new runs against reference
- Flag any deviations

What to Compare:

- Calibrated parameters
- NPV values (within MC noise)
- Exercise probabilities
- Greeks (if computed)

When to Update Reference:

- Bug fixes that change results
- Model enhancements
- After thorough validation

Performance Testing

Benchmarks:

- Full pricing: <15 seconds
- 30,000 paths, 10Y horizon
- MacBook M1/M2 baseline

Bottlenecks:

- ① Path simulation (main loop)
- ② Cash flow accumulation
- ③ Regression (matrix operations)

Optimization Strategies:

- Vectorize all operations
- Minimize memory allocation
- Use efficient BLAS (via NumPy)
- Consider chunking for large N

Appendix E

Practical Considerations

Real-World Deployment Challenges

Market Data Challenges

Data Sources:

- Bloomberg, Reuters
- Exchange feeds
- Broker quotes

Data Quality Issues:

- Stale quotes
- Bid/ask spreads
- Missing data points
- Timezone differences

Our Approach:

- JSON input files
- Placeholder/illustrative data
- Clear “REPLACE” markers
- Schema validation

Collateral and Funding

CSA (Credit Support Annex):

- Determines collateral posting rules
- Affects discount curve choice
- USD CSA $\Rightarrow$ SOFR discounting

Cross-Currency Collateral:

- EUR deal, USD collateral
- Creates “effective” EUR curve
- Different from native EUR curve

Our Implementation:

- Build EUR effective curve from FX forwards
- Discount EUR flows at effective rate
- Maintain forecast/discount separation

Model Risk Management

Key Risks:

- Calibration failure
- Parameter instability
- Numerical errors
- Assumption violations

Mitigations:

- Tolerance thresholds
- Calibration acceptance flags
- Martingale diagnostics
- Limitation documentation

Documentation:

- Model specification
- Validation report
- Known limitations
- Appropriate use guidance

Greeks and Risk

Bump-and-Revalue:

- Shift input, reprice, compute difference
- Simple but expensive
- Need common random numbers

Key Greeks:

- **Delta:** $\partial V / \partial \text{rates}$
- **Vega:** $\partial V / \partial \text{vol}$
- **FX Delta:** $\partial V / \partial X$
- **Correlation:** $\partial V / \partial \rho$

Not Implemented Yet:

- Pathwise derivatives
- Likelihood ratio method
- AAD (automatic differentiation)

Performance Optimization

Current Performance:

- 30K paths: ~10 seconds
- 100K paths: ~30 seconds
- Acceptable for research

Potential Improvements:

- ① **Numba JIT:** Compile hot loops
- ② **CuPy:** GPU acceleration
- ③ **Cython:** C extensions
- ④ **C++ Port:** Full rewrite

Trade-offs:

- Development time vs runtime
- Maintainability
- Debugging complexity

Production Readiness Checklist

Before Production Use:

- ☐ Independent model review
- ☐ Dual/upper bound implementation
- ☐ Full market data integration
- ☐ Greeks computation
- ☐ Audit trail
- ☐ Error handling review
- ☐ Performance testing
- ☐ Security audit
- ☐ Documentation complete
- ☐ Regulatory approval

Current Status: Research/educational use only

Lessons Learned

- ① **Start Simple:** Build vanilla first, then exotic
- ② **Validate Early:** Test each component before integration
- ③ **Document Assumptions:** Future you will forget
- ④ **Use QuantLib:** Don't reinvent dates, calendars, curves
- ⑤ **Vectorize:** NumPy is your friend
- ⑥ **Test Extensively:** Martingales, bounds, convergence
- ⑦ **Accept Limitations:** No model is perfect

Resources and References

Books:

- Brigo & Mercurio: “Interest Rate Models”
- Glasserman: “Monte Carlo Methods in Financial Engineering”
- Longstaff & Schwartz (2001): LSM Paper

QuantLib:

- <https://www.quantlib.org>
- <https://github.com/lballabio/QuantLib>
- <https://quantlib-python-docs.readthedocs.io>

This Project:

- Repository: `tgir_quantlib_tools`
- Model ID: `HW1F_USD_HW1F_EUR_BSFX_LSM_V1`

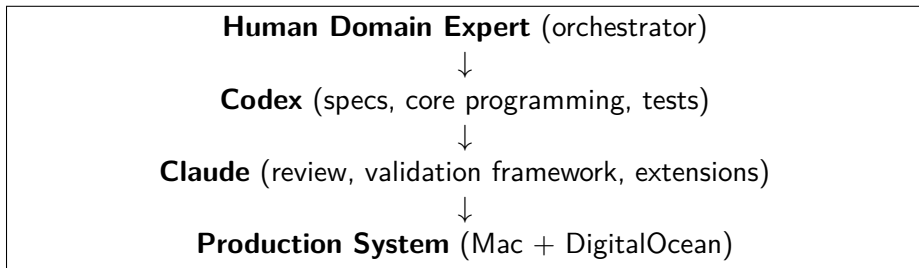
Appendix F

Collaborative AI Development

Human + Codex + Claude + Tools

The Collaborative Development Model

This project demonstrates a new development paradigm:



Key Insight:

The collaborative methodology is **as significant as the callable XCCY pricer itself**

Reproducible Pattern: Same workflow can build other complex pricers

Division of Labor: Two AI Agents

	Codex (OpenAI)	Claude (Anthropic)
Primary Role	Core development	Review & validation
Specs	Wrote formal specifications	Reviewed for completeness
Implementation	Built Monte Carlo engine	Reviewed for correctness
Tests	Created initial test suite	Built validation framework
Extensions	—	Extended with regression tests

Human Role:

- Provided domain expertise (financial mathematics)
- Orchestrated agent collaboration
- Final validation against theory
- Verified with Stack Overflow guidance

Codex: Ground-Up Development

What Codex Built from Human Guidance:

① Rigorous Specifications:

- Mathematical model documentation (LaTeX)
- API contracts with type hints
- JSON schemas for market data and trades

② Core Implementation:

- Three-factor Monte Carlo (1,400+ lines)
- Longstaff-Schwartz regression
- SOFR/ESTR curve bootstrapping
- QuantLib integration layer

③ Initial Test Suite:

- Unit tests for each component
- Integration tests for pricing
- Smoke tests for API

Human provided: Math formulas, market conventions, QuantLib patterns

Claude: Review and Validation Framework

Claude's Key Contributions:

① Code Review:

- Reviewed all Codex implementations
- Identified numerical stability issues
- Suggested vectorization optimizations

② Validation Framework (Critical):

- Martingale test infrastructure
- Structural bounds verification
- Monotonicity checking
- Convergence analysis

③ Extensions for Regression:

- Built tests to catch future changes
- Automated validation on every commit
- Documentation and lecture generation

Result: 20 automated tests that provide confidence in correctness

The Full Technology Ecosystem

Layer	Component	Role
Infrastructure	Mac (local)	Development environment
	DigitalOcean	Production hosting
AI Agents	Codex (OpenAI)	Specs, core dev, tests
	Claude (Anthropic)	Review, validation, docs
	GitHub Copilot	Inline VS Code assistance
Libraries	QuantLib (C++)	Curves, calibration
	Python/NumPy	Simulation, Flask
Resources	Stack Overflow	Guidance verification

Cohesion: All components integrated in unified workflow

Integration Boundaries: Proposed REST + Implemented MCP

Two Ways to Access the Pricer:

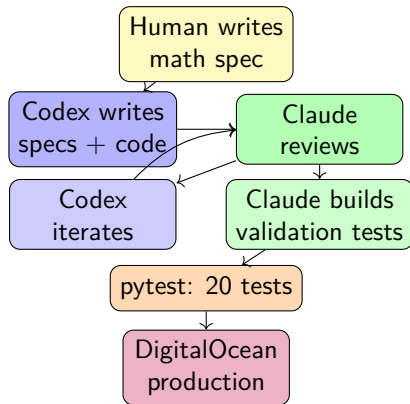
Interface	Use Case	Protocol
REST contract	Future trading system integration (C++ clients, batch jobs)	HTTPS + JSON Proposed /api/v1/ endpoints
MCP Server	LLM integration (Claude, GPT can call pricer)	JSON-RPC stdio or HTTP

Specified REST Endpoints (not yet implemented):

- POST /api/v1/callable-xccy/price — Sync pricing
- POST /api/v1/callable-xccy/jobs — Async job submission
- GET /api/v1/engine — Schema and limits

Key Principle: No QuantLib/Python types cross API boundary

Collaborative Workflow Diagram



Iteration: Multiple cycles until all 20 tests pass

Zero-Intervention Automation Pipeline

A Hobby Project with Production-Grade Rigor:

One prompt triggers the **entire pipeline** — no human intervention:

- 1 **Coding** — AI implements from specs
- 2 **Regression Testing** — pytest runs 20+ tests locally
- 3 **Browser UI Testing** — Playwright validates Flask UI
- 4 **Documentation** — LaTeX auto-updated
- 5 **Deploy to DigitalOcean** — SSH + Git + systemd
- 6 **Production Regression** — Tests on live server
- 7 **Telegram Notification** — “Done!”

Key Insight: \$7/month hobby project built with the same rigor as top tech company production systems.

QuantLib Extension Strategy

QuantLib's Limitations:

- No native callable XCCY support
- Limited multi-currency Monte Carlo
- No built-in LSM for cross-currency

How We Extended It (Collaboratively):

- ① **Human:** Defined what QuantLib provides vs what's needed
- ② **Codex:** Wrote custom Monte Carlo using QuantLib curves
- ③ **Claude:** Verified QuantLib API usage against docs
- ④ **Stack Overflow:** Confirmed edge cases and patterns

Result:

- Use QuantLib for: curves, dates, calendars, basic calibration
- Custom code for: multi-factor MC, LSM, XCCY cashflows
- 1,400+ lines of rigorous extensions

DigitalOcean Deployment: Detailed Steps

1. Droplet Creation (DigitalOcean Console):

- Region: NYC3 or SFO3 for US users
- Image: Ubuntu 22.04 LTS
- Plan: Basic \$7/month (2GB RAM, 1 vCPU) — hobby-friendly!
- Authentication: SSH key (not password)

2. Initial Server Setup:

```
ssh root@your-droplet-ip
apt update && apt upgrade -y
adduser deploy
usermod -aG sudo deploy
```

3. Application Deployment:

```
cd /opt && git clone <repo>
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
cp deploy/systemd/*.template /etc/systemd/system/
systemctl enable --now tgir-quantlib-tools
```

DigitalOcean: SSL and Apache Configuration

Apache Setup:

```
apt install apache2
a2enmod proxy proxy_http ssl rewrite
cp deploy/apache/*.template /etc/apache2/sites-available/
a2ensite tgir-quantlib-tools
systemctl reload apache2
```

SSL with Certbot:

```
apt install certbot python3-certbot-apache
certbot --apache -d your-domain.com
```

Firewall:

```
ufw allow 22/tcp
ufw allow 80/tcp
ufw allow 443/tcp
ufw enable
```

Result: HTTPS-secured Flask app at <https://your-domain.com>

Why Codex + Claude Collaboration Works

Complementary Strengths:

- **Codex:** Excellent at ground-up implementation from specs
- **Claude:** Excellent at review, validation, extension
- **Human:** Domain expertise, orchestration, final judgment

Error Detection:

- Claude catches bugs in Codex's implementations
- Codex fixes issues Claude identifies
- Neither alone would produce same quality

Stack Overflow Role:

- AI can hallucinate APIs that don't exist
- Human experts catch edge cases
- Community validates approaches

Key Lesson:

Two AI agents + human domain expert
produces better results than any single agent

The Methodology IS the Innovation

This project delivers TWO things:

① Callable XCCY Pricer:

- Three-factor Monte Carlo
- Longstaff-Schwartz optimization
- 20 validation tests
- Production deployment

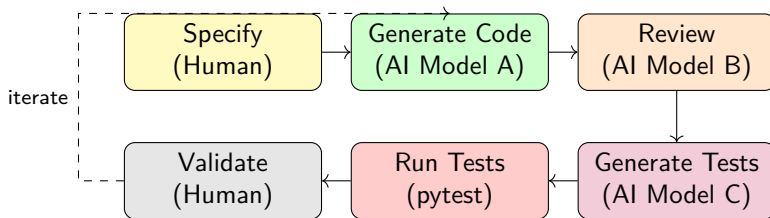
② Collaborative Development Methodology:

- Human → Codex (specs, implementation)
- Codex → Claude (review, validation)
- Claude → Regression tests (future-proofing)
- Full ecosystem: Mac + DigitalOcean + QuantLib + Python

The methodology is reusable for other complex pricers

The collaborative workflow is as valuable as the callable XCCY pricer itself

Reusable Human-Agent Development Pattern



Key Principle:

Human provides specifications and validation
AI accelerates implementation and review

End of Supplementary Material

Return to Main Presentation